

Domotics - OS Project

Boscardin Denise - 243923

Cappelletti Marco - 245089

August 27, 2026

1 Introduction

The project implements an emulated home automation system, in which each device (bulb, window, fridge, hub, or timer) is represented by a distinct UNIX process, spawned by the Controller, our root process. Processes communicate through POSIX named pipes (FIFOs), exchanging messages according to a protocol defined in `protocol.h`.

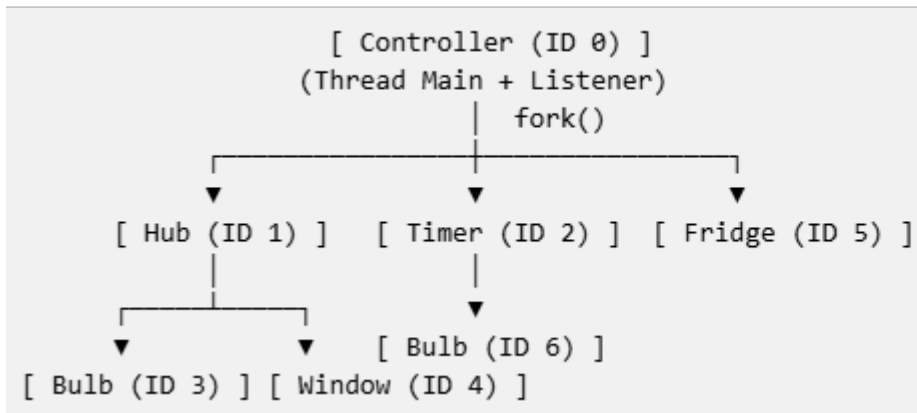
In this report we describe our implementation choices and the motivations behind them.

2 System Architecture

In our implementation, the Controller is the root process and is the only process that calls `fork()`. All physical and logical devices are therefore created as direct children of the Controller, in a flat OS-level hierarchy. Each device runs independently in its own process and has its own event loop.

We handle the logical hierarchy of the house separately, using a single FIFO per process for both listening and writing, rather than a dedicated read/write pair.

The Controller is the root process and has ID 0 by convention: it is unique and cannot be deleted.



Each process creates a dedicated FIFO (`/tmp/domotica_<type>_<id>.fifo`) and continuously listens on it in a non-blocking cycle.

2.1 Processes Hierarchy

When a new device is created by the user, the Controller calls `fork()` to spawn the corresponding device process.

2.2 Device types

We have two (purely logical) categories: Control Devices (Controller, Hub, Timer) and Interaction Devices (Bulb, Window, Fridge).

Device	Type	State/Registries	Role
Controller	Control	on/off, num	Unique and not erasable, OS root. ID 0
Hub	Control	mirrors children state	Propagates switch and delete on cascade
Timer	Control	begin, end	Turns on/off based on the begin/end registry
Bulb	Interaction	on/off, time	
Window	Interaction	open/closed, time	
Fridge	Interaction	open/closed, perc, temp, thermostat, delay	

3 Design choices

3.1 IPC mechanisms

Communication between processes is implemented using dedicated POSIX named pipes (FIFOs). Our alternatives were shared memory, message queues, and sockets. We chose FIFOs for three reasons:

- They are simple to use, relying only on standard `read()/write()` calls.
- They can be written to directly from a Bash command, which is essential for external manual interaction.
- They expose a filesystem endpoint (`/tmp/domotica_<type>_<id>.fifo`), so the `manual_interaction.sh` script can reach a device without going through the Controller.

Hubs and Timers use a mirroring mechanism when processing an INFO command. Instead of relying only on their local state, they query their connected children for their current information, and the parent then combines the received results. For example, if different children report different states, such as one device being On and another Off, the parent reports the resulting state as `STATUS_MANUAL_OVERRIDE`.

During development, we also found an issue related to how FIFOs handle data. A FIFO is a byte stream and does not preserve message boundaries. Therefore, when two commands such as `CMD_SET_PARENT` and `CMD_DELETE` were sent immediately one after the other, the child could read them together instead of processing them separately. In our implementation, this could cause the child to stop responding correctly. We solved this problem by adding a short delay between the two commands in the Controller. Specifically, we use `usleep(100000)` so that the first command has time to be processed before the next one is sent.

3.2 Concurrency management

The Controller is multithreaded (using `pthread`): a listener thread receives all messages on the Controller's FIFO, while the main thread manages the interactive shell. Synchronization is implemented with a mutex and condition variable pair:

```

1 pthread_mutex_lock(&mutex);
2 response_received = false;
3 ipc_send_message(fd, command);
4 while (!response_received)
5     pthread_cond_wait(&sync_cond, &mutex);
6 pthread_mutex_unlock(&mutex);

```

Every device uses non-blocking I/O with polling (a `usleep(50000)` call when the FIFO is empty) instead of `select()`, because it only ever needs to watch one file descriptor: its own FIFO. Using `select()` to monitor a single fd would just add unnecessary complexity.

This mechanism makes an inherently asynchronous IPC exchange look synchronous to the user: the main thread waits until the listener thread signals that the reply has arrived, so the shell only prints the next prompt after the device has actually responded.

We handle `SIGINT` only in the Controller. Child processes ignore the signal by setting `SIGINT` to `SIG_IGN`. When the Controller receives `SIGINT`, it starts the cleanup process: it stops the running threads, sends `SIGTERM` to all child processes, and waits for them using `waitpid()`. After all processes have terminated, it removes the FIFOs created in `/tmp`.

There's also a random 1-3s delay for simulating the processing time, through `ipc_simulate_delay()` function.

3.3 State consistency and manual override

Hub and Timer do not keep a persistent state: they rebuild it on every INFO request by querying their children (through the `get_state()/get_timer_state()` functions).

The Override Resolution Rule is implemented as follows: the aggregate state is `STATUS_MANUAL_OVERRIDE` whenever the children's states are inconsistent (e.g., due to a manual override on one of them); if all children report a coherent state, the aggregate state is simply Active or Inactive. As soon as the parent Control Device receives a new command, it propagates it to all children and clears any previously reported manual override, exactly as required by the specification.

3.4 Routing and logical linking

The link `<id1> to <id2>` command follows a sequence of checks:

- Refuses a self-link (`ERR_SELF_LINK`) and verifies that both IDs exist (`ERR_DEVICE_NOT_FOUND`).
- Checks that `id2` is a Control Device (`ERR_DEVICE_TYPE_MISMATCH` otherwise).
- Checks the absence of cycles (`creates_cycle()`, `ERR_CYCLE_DETECTED`).
- If `id1` was already linked to another parent, it is unlinked automatically first.
- Sends `LINK <id1> <type>` to the new parent; once it replies `STATUS_OK`, sends `SET_PARENT <id2>` to the child and updates the routing tables accordingly.

4 Error handling

All IPC operations return a numeric status code, defined in `protocol.h`. The same codes are used in `manual_interaction.sh`.

Code	Value	Meaning
<code>STATUS_OK</code>	0	Successful operation.
<code>ERR_DEVICE_NOT_FOUND</code>	1	Non-existing ID.
<code>ERR_INVALID_COMMAND</code>	2	Command not recognized.
<code>ERR_LINK_FAILED</code>	3	Failed link.
<code>ERR_DEVICE_TYPE_MISMATCH</code>	4	Target is not a Control Device.
<code>STATUS_MANUAL_OVERRIDE</code>	5	Inconsistent state due to manual override.
<code>ERR_DEVICE_CRASHED</code>	6	Child process terminated unexpectedly.
<code>ERR_INVALID_PARAM</code>	7	Invalid parameter.
<code>ERR_CYCLE_DETECTED</code>	8	The link would create a cycle.
<code>ERR_NOT_FOUND</code>	9	Invalid entity.
<code>ERR_SELF_LINK</code>	10	Device linked to itself.
<code>ERR_TIMEOUT</code>	11	Timeout while waiting for a reply.
<code>ERR_RESOURCE_ERROR</code>	12	Resource error (fork, FIFO, memory).
<code>ERR_PERMISSION_ERROR</code>	13	Forbidden operation.

Validation is handled on two levels: on the Controller side, before sending the IPC command, to reject clearly invalid input; and on the device side, for example for numeric errors, invalid flags, or system errors.

5 Edge case handling

- **Manual override and Controller command issued simultaneously:** both channels write to the same FIFO, but every `write()` is atomic, so one message is always processed before the other.
- **Sudden device crash (SIGKILL):** detected by `handle_sigchld()` in the Controller (`waitpid(WNOHANG)` in a loop), which removes the device from `devices[]`, closes its FIFO, and sends `CHILD_DIED <id>` to the logical parent if needed.
- **Interaction Device linked as a parent:** refused upstream in `link_devices()` with `ERR_DEVICE_TYPE_MISMATCH`.

- **Self-link or cycle:** a self-link is refused immediately (`ERR_SELF_LINK`). Cycles are detected by the `creates_cycle()` function.
- **Deleting a Hub with children:** cascade deletion. The Hub propagates `DELETE` to all its children, recursively.
- **Switch command sent to a Hub with a crashed child:** the Hub tries to open the dead child's FIFO but fails with `ENXIO`. A timeout prevents the Hub from getting stuck.
- **Invalid begin/end for the Timer:** the Timer validates both the `HH:MM` format and the logical coherence (`begin < end`) before accepting the change. Schedules crossing midnight (e.g., `begin=22:00`, `end=02:00`) are not supported.

For manual interaction with the devices, we implemented the `manual_interaction.sh` Bash script. The script checks the user input and writes commands directly to the FIFOs of the selected devices. We distinguish manual commands from normal Controller commands by adding the `MANUAL` keyword to the IPC message. This is important because some hardware registers (`perc` and `thermostat` in Fridge) can only be modified through manual interaction. Devices check this information before applying a modification and reject requests coming directly from the Controller when they are not authorized.

6 Execution example

```

1 $ make run
2 > add hub # ID 1
3 > add timer # ID 2
4 > add bulb # ID 3
5 > add window # ID 4
6 > add fridge # ID 5
7 > add bulb # ID 6
8 > link 3 to 1 # Bulb 3 is child of Hub 1
9 > link 4 to 1 # Window 4 is child of Hub 1
10 > link 6 to 2 # Bulb 6 is child of Timer 2
11 > switch 2 begin 22:00 # sets the Timer
12 > switch 2 end 23:00
13 > switch 1 power on # propagates on cascade to Bulb 3 and Window 4
14 > switch 5 open on # opens the fridge and starts time tracking
15 > switch 5 delay 30 # in 30 seconds the fridge will auto-close
16 > info 1 # Hub aggregate state (children mirroring)
17 > info 2
18 > info 5
19 > list # all devices list, with PID and logical link

```

7 Conclusions

The project implements a multi-process system where the separation between the OS hierarchy (flat) and the logical hierarchy (dynamic, IPC-based) allows control relationships between devices to be modified at runtime without terminating or restarting processes. POSIX FIFOs proved to be the most suitable tool for combining a simple messaging interface with the need for an endpoint directly accessible from an external Bash script. This mutex/condition-variable mechanism lets the shell stay simple to use for the operator, while the system keeps reacting to asynchronous events such as crashes, Timer notifications, and manual overrides.